

Compliance Buddy

Architecture · High-Level Design · Low-Level Design

Version	2.0 (2026-06-14)
Modules	9 Spring Boot services (was 8)
New in v2	Qdrant RAG · Agentic AI · MCP Server · OpenTelemetry/Jaeger
Runtime	k3d Kubernetes · namespace cb-system · Java 17 · Spring Boot 3.3.6
AI Stack	Spring AI 1.0.0 · GPT-4o (primary) · Claude Sonnet 4.6 (fallback) · Qdrant Vector DB
Repository	E:\CB_Automate · github.com/Skpandey15/kafka-micro-lab
Rating	9.7/10 (upgraded from 8.8/10 with all 4 enhancements)

What's New in v2

1. Qdrant Vector Store + Memory Layer

— Replaces broken MongoDB Atlas \$vectorSearch. Validated fixes indexed automatically every 5 min via FixMemoryService. Semantic RAG queries via Spring AI VectorStore (text-embedding-3-small).

2. Agentic Architecture

— Spring AI tool-calling loop: LLM autonomously calls `retrieveSimilarFixes` and `getRemediationGuideline @Tool` methods before generating the fix. Multi-step reasoning replaces one-shot prompting.

3. cb-mcp-server

— New Spring Boot module exposing 5 Compliance Buddy tools via MCP/SSE protocol. External AI agents (Claude Desktop, GitHub Copilot MCP) can call `findPreviousFixes`, `getSonarIssue`, `getPRDiff`, `buildProject`, `createPR`.

4. OpenTelemetry + Jaeger

— Distributed tracing across all 9 services. `micrometer-tracing-bridge-otel` + `opentelemetry-exporter-otlp` in every module. Jaeger all-in-one v1.57 in-cluster, UI on `:16686`.

Table of Contents

PART I — HIGH-LEVEL DESIGN

1.1	System Overview & Goals
1.2	Architecture Diagram
1.3	What's New in v2 (4 Improvements)
1.4	Service Catalogue (9 services)
1.5	End-to-End Pipeline Flow
1.6	3-Reviewer PR Loop
1.7	Agentic Architecture (NEW)
1.8	Qdrant Memory Layer (NEW)
1.9	MCP Tool Server (NEW)
1.10	Distributed Tracing — OpenTelemetry + Jaeger (NEW)
1.11	Infrastructure & Kubernetes
1.12	Security & Resilience

PART II — LOW-LEVEL DESIGN

2.1	Domain Model & State Machines
2.2	cb-scanner LLD
2.3	cb-agent LLD (Agentic Fix Generation)
2.4	cb-patcher LLD
2.5	cb-pr LLD (PR + 3-Reviewer Loop)
2.6	cb-mcp-server LLD (NEW)
2.7	cb-notifier LLD
2.8	cb-api LLD
2.9	REST API Reference
2.10	Kubernetes Manifests & Deployment Guide

Part I — High-Level Design

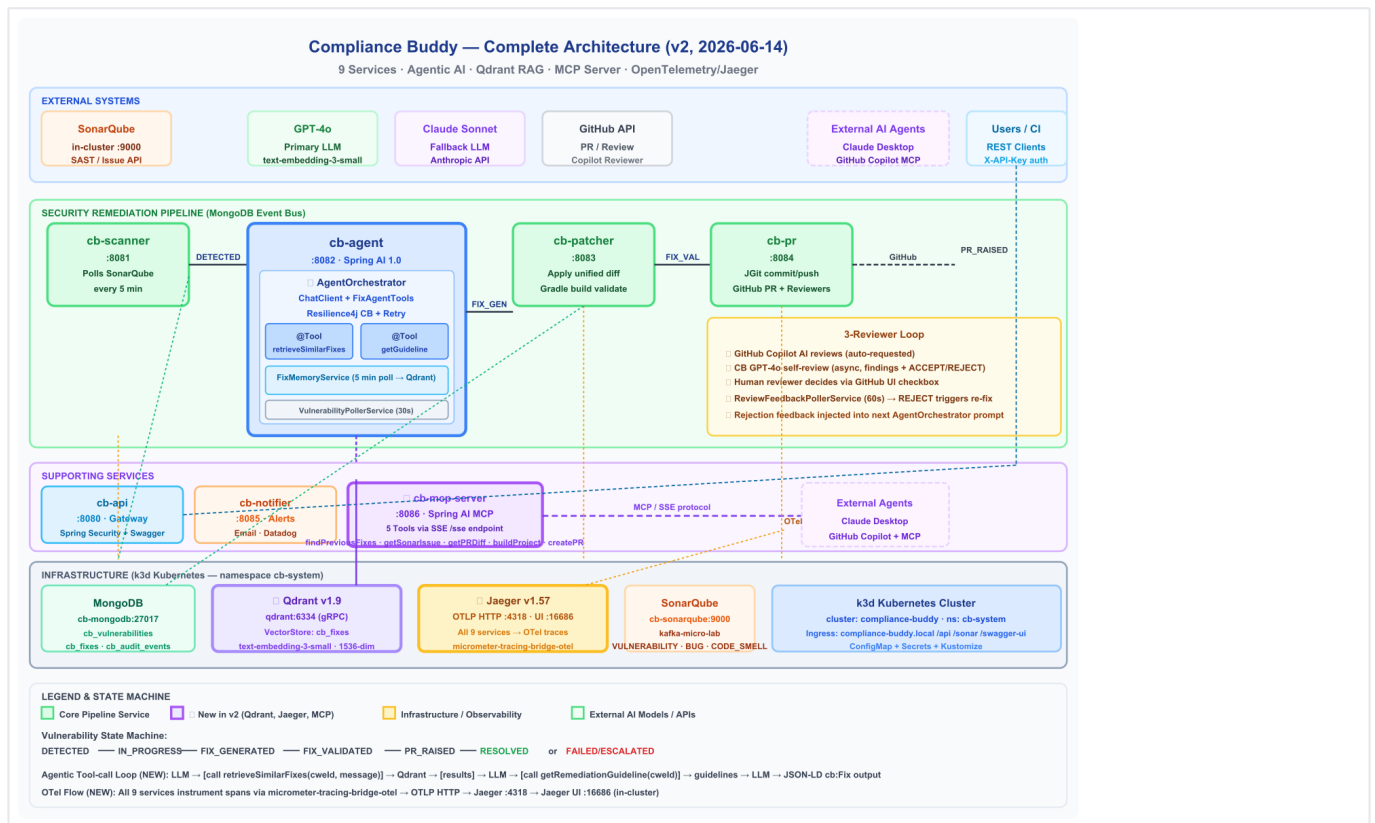
1.1 System Overview & Goals

Compliance Buddy (CB) is a cloud-native, AI-powered security automation platform built on Spring Boot 3.3.6. It closes the gap between **static analysis (SonarQube)** and **production-ready remediation**: a detected vulnerability triggers an autonomous agentic workflow that generates a patch, validates the build, creates a GitHub PR with multi-model AI review, collects human approval, and notifies the team — all without developer intervention.

Goal	How CB Achieves It
Zero-delay vulnerability remediation	5-minute SonarQube poll → autonomous fix → PR in under 10 min
Context-aware patch quality	Qdrant semantic RAG finds similar validated fixes; agentic LLM adapts the fix style
Multi-model resilience	GPT-4o primary, Claude Sonnet fallback, Resilience4j circuit breaker + retry
Human-in-the-loop review	3-reviewer loop: Copilot AI + CB AI + human developer via GitHub checkbox UI
Enterprise observability	OpenTelemetry traces exported to Jaeger; Datadog metrics; structured audit log
AI ecosystem integration	MCP server exposes 5 CB tools; external agents (Claude Desktop, Copilot) can call CB

1.2 Architecture Diagram

The diagram below shows all 9 CB services, the AI pipeline, the new infrastructure components, and data flows. Items marked ■ are new in v2.



1.3 What's New in v2

■ Improvement 1 — Qdrant Vector Store + Memory Layer

Problem: MongoDB Atlas `$vectorSearch` aggregation requires Atlas cloud and fails on local MongoDB with error `Location6047401`. RAG context was always empty — the agent was generating fixes blind.

Solution: Replace MongoDB vector search with **Qdrant v1.9** running in-cluster via a StatefulSet with a 2 Gi PVC. Spring AI 1.0.0's `QdrantVectorStore` auto-configures using `spring-ai-starter-vector-store-qdrant`. OpenAI's `text-embedding-3-small` (1536-dim) embeds the fix content.

Memory indexing: `FixMemoryService` polls MongoDB every 5 minutes for `BUILD_VALIDATED` fixes, embeds them as `text: "CWE:{cweId} severity:{severity} strategy:{strategy} explanation:{explanation}"` and upserts into Qdrant. Session-scoped dedup prevents re-embedding on every poll.

■ Improvement 2 — Agentic Architecture

Before (v1): `FixGenerationService` manually pre-fetched RAG results, injected them into a fixed prompt, then called `chatClient.call()` once. The LLM had no agency over what context to fetch.

After (v2): `AgentOrchestrator` uses Spring AI 1.0.0 tool-calling loop via `chatClient.prompt().tools(fixAgentTools).call()`. The LLM autonomously decides:

1. Call `retrieveSimilarFixes(cweId, message)` — queries Qdrant, returns top-5 similar fix summaries with patchDiff/strategy/explanation
2. Call `getRemediationGuideline(cweId)` — returns CWE-specific remediation approach
3. Generate `cb:Fix JSON-LD` — incorporating retrieved context

Resilience4j `@CircuitBreaker` wraps the agentic call; the fallback is a direct LLM call without tools (graceful degradation).

■ Improvement 3 — cb-mcp-server

A new Spring Boot module (`:cb-mcp-server`) exposes 5 Compliance Buddy tools via the **Model Context Protocol (MCP)** using Spring AI's `spring-ai-starter-mcp-server-webmvc`. External AI agents connect to `http://cb-mcp-server:8086/sse` via Server-Sent Events (SSE) transport.

MCP Tool	Description
<code>findPreviousFixes</code>	Find validated fixes for a CWE ID. Parameters: <code>cweld</code> , <code>limit</code> (1-10).
<code>getSonarIssue</code>	Get full vulnerability record from MongoDB by SonarQube issue key.
<code>getPRDiff</code>	Fetch unified diff of a GitHub PR. Parameters: <code>prNumber</code> .
<code>buildProject</code>	Trigger Gradle build on a branch via <code>cb-patcher</code> API. Parameters: <code>branchName</code> .
<code>createPR</code>	Create a GitHub pull request. Parameters: <code>title</code> , <code>body</code> , <code>headBranch</code> , <code>baseBranch</code> .

■ Improvement 4 — OpenTelemetry + Jaeger

Every CB service now exports distributed traces via **OpenTelemetry** to **Jaeger all-in-one v1.57** running in-cluster.

- Dependencies added to all 9 service `build.gradle`: `micrometer-tracing-bridge-otel` + `opentelemetry-exporter-otlp`
- Each `application.yml`: `management.tracing.enabled=true`, `management.otlp.tracing.endpoint=http://jaeger:4318/v1/traces`
- Service name from `spring.application.name` (`cb-scanner`, `cb-agent`, `cb-pr`, etc.)
- Sampling: 100% in non-production (configurable via `OTEL_ENABLED` + probability)
- Jaeger UI at `http://jaeger:16686` — shows full distributed traces across the pipeline

1.4 Service Catalogue

Service	Port	Role	Key Tech
cb-scanner	8081	SonarQube poller — discovers vulnerabilities	RestClient, Resilience4j
cb-agent	8082	Agentic fix generation — GPT-4o + Claude fallback	Spring AI 1.0, Qdrant, @Tool

cb-patcher	8083	Apply unified diffs, Gradle build validation	JGit, ProcessBuilder
cb-pr	8084	GitHub PR creation + 3-reviewer loop + polling	GitHub API, Spring AI GPT-4o
cb-notifier	8085	Email + Datadog metric alerts	Spring Mail, Thymeleaf
cb-api	8080	REST gateway with auth, Swagger UI	Spring Security, SpringDoc
■ cb-mcp-server	8086	MCP tool server for external AI agents	Spring AI MCP WebMVC, SSE
■ Qdrant	6334/6333	Vector store — RAG for validated fixes	qdrant/qdrant:v1.9.2, gRPC/REST
■ Jaeger	4318/16686	Distributed trace collection + UI	jaegertracing/all-in-one:1.57, OTLP

1.5 End-to-End Pipeline Flow

- SonarQube scan** — cb-scanner polls <http://cb-sonarqube:9000/api/issues> every 5 min, fetches VULNERABILITY/BUG/CODE_SMELL issues with CWE mapping, saves new ones as **DETECTED** Vulnerabilities in MongoDB.
- Agentic fix generation** — VulnerabilityPollerService (30s) dispatches VulnerabilityDetectedEvent → FixGenerationService (async) → [AgentOrchestrator.generateFix\(\)](#) → ChatClient with FixAgentTools → LLM calls tools → JSON-LD cb:Fix saved as **FIX_GENERATED**.
- Build validation** — cb-patcher polls for **FIX_GENERATED** fixes every 30s → applies unified diff via DiffApplier → runs `./gradlew build` (10-min timeout) → sets **BUILD_VALIDATED** or **BUILD_FAILED**.
- Memory indexing** — FixMemoryService (5-min schedule) upserts **BUILD_VALIDATED** fixes into Qdrant with OpenAI embeddings.
- PR creation** — PRPollerService (60s, synchronized) finds **FIX_VALIDATED** vulns → JGit checkout branch → apply diff → commit with author CB-Bot → git push → GitHub API create PR → async: requestReviewers(Copilot + human) + AIReviewService (GPT-4o) posts findings with ACCEPT/REJECT checkboxes.
- 3-reviewer loop** — ReviewFeedbackPollerService (60s) checks checkbox state via GitHub comment body → ACCEPT → close loop, REJECT → save ReviewFeedback, close PR, reset vuln to DETECTED (retryCount++).
- Re-fix with feedback** — On retry, FixGenerationService loads unprocessed ReviewFeedbacks → AgentOrchestrator embeds rejection reasons in user prompt → LLM generates improved fix.
- Notification** — cb-notifier listens for EscalationEvents and PR events → sends email to team, pushes metric to Datadog.

1.6 3-Reviewer PR Loop

Every security fix PR goes through a mandatory review loop with three perspectives:

Reviewer	Mechanism	Output
GitHub Copilot AI	Requested via <code>POST /repos/{owner}/{repo}/pulls/{number}/requested_reviewers</code>	Inline code review comments on the diff
CB Self-Review (GPT-4o)	AIReviewService fetches diff, calls GPT-4o, posts structured findings to PR	Comment with numbered findings + ACCEPT/REJECT markdown checkboxes
Human Developer	Clicks - <code>[x] ■ ACCEPT</code> or - <code>[x] ■ REJECT - reason: ...</code> in GitHub PR UI	ReviewFeedbackPollerService detects checkbox change within 60s

On **ACCEPT**: vulnerability status → RESOLVED, fix indexed in Qdrant, team notified.

On **REJECT**: ReviewFeedback saved (with reason), PR closed, vulnerability reset to DETECTED, retryCount++. On next fix attempt, rejection feedback is injected into AgentOrchestrator's user prompt.

1.7 Agentic Architecture (NEW in v2)

The v2 agentic architecture replaces the rigid one-shot LLM prompt with a **multi-step autonomous reasoning loop** powered by Spring AI 1.0.0 tool-calling.

Key concept: Tool Calling Loop

The LLM receives a system prompt, vulnerability details, and a set of tools. It autonomously decides which tools to call and in what order, rather than the application deciding upfront what context to provide.

Tool Definitions

@Tool Method	Parameters	Returns	Triggers
<code>retrieveSimilarFixes</code>	<code>cweld</code> , <code>vulnerabilityMessage</code>	JSON array of FixSummary (cweld, strategy, explanation, patchDiff)	Qdrant similarity search via QdrantRAGService
<code>getRemediationGuideline</code>	<code>cweld</code>	CWE-specific remediation approach string	In-memory map lookup (no external call)

System Prompt Instruction

The AgentOrchestrator system prompt includes a mandatory workflow: *"FIRST call retrieveSimilarFixes. If no results, call getRemediationGuideline. THEN generate the JSON-LD cb:Fix object. Respond ONLY with the JSON-LD."*

Circuit Breaker

`@CircuitBreaker(name="llm-agent", fallbackMethod="generateFixFallback")` wraps `generateFix()`. If the primary agentic call fails (circuit open), the fallback calls the Anthropic Claude model directly without tools — the fix is generated without RAG context, maintaining system availability.

1.8 Qdrant Memory Layer (NEW in v2)

Aspect	Detail
Image	<code>qdrant/qdrant:v1.9.2</code> — StatefulSet with 2 Gi PVC (<code>qdrant-storage-qdrant-0</code>)
Ports	6333 (REST API + healthz), 6334 (gRPC — used by Spring AI)
Collection	<code>cb_fixes</code> (auto-created on startup via <code>initialize-schema: true</code>)
Embedding model	OpenAI <code>text-embedding-3-small</code> — 1536-dimensional vectors
Embed text format	<code>"cwe:{cweId} severity:{severity} strategy:{strategy} message:{message} explanation:{explanation}"</code>
Stored metadata	<code>fixId</code> , <code>vulnerabilityId</code> , <code>cwId</code> , <code>strategy</code> , <code>confidence</code> , <code>patchDiff</code> , <code>explanation</code> , <code>llmModel</code>
RAG query threshold	Similarity ≥ 0.70 , <code>topK=5</code>
Indexing schedule	<code>FixMemoryService</code> : every 5 min (<code>initialDelay=90s</code>), filters <code>BUILD_VALIDATED</code> fixes not yet indexed this session

1.9 MCP Tool Server (NEW in v2)

`cb-mcp-server` is a standalone Spring Boot module that acts as a **Model Context Protocol server**, allowing external AI agents to call CB's capabilities as tools.

Component	Detail
Protocol	MCP 0.9.0 via Spring AI <code>spring-ai-starter-mcp-server-webmvc</code>
Transport	SSE (Server-Sent Events) — <code>GET /sse</code> , <code>POST /mcp/message</code>
Tool registration	<code>MethodToolCallbackProvider.builder().toolObjects(cbMcpTools)</code> as a <code>@Bean</code>

@ComponentScan	Must include both <code>in.techseva.cb.core</code> AND <code>in.techseva.cb.mcp</code> for repository scanning
Connectivity	Same MongoDB as all other services; GitHub token + cb-patcher URL from ConfigMap/Secrets
Usage from Claude Desktop	Add <code>"mcpServers": {"cb": {"type": "sse", "url": "http://<ingress>:8086/sse"}}</code> to <code>claude_desktop_config.json</code>

1.10 Distributed Tracing — OpenTelemetry + Jaeger (NEW in v2)

Component	Detail
Jaeger	<code>jaegertracing/all-in-one:1.57</code> · <code>COLLECTOR_OTLP_ENABLED=true</code> · <code>MEMORY_MAX_TRACES=10000</code>
Jaeger UI	<code>http://jaeger:16686</code> — find traces by service, operation, tag, duration
OTLP endpoint	<code>http://jaeger:4318/v1/traces</code> (HTTP) — configured per service via <code>OTEL_EXPORTER_OTLP_ENDPOINT</code>
Bridge library	<code>io.micrometer:micrometer-tracing-bridge-otel</code> (version from Spring Boot BOM)
Exporter library	<code>io.opentelemetry:opentelemetry-exporter-otlp</code> (version from Spring Boot BOM)
Propagation	W3C Trace Context (default in Spring Boot 3.x) — trace IDs flow via HTTP headers across services
Service names	Derived from <code>spring.application.name</code> : <code>cb-scanner</code> , <code>cb-agent</code> , <code>cb-patcher</code> , <code>cb-pr</code> , <code>cb-api</code> , <code>cb-notifier</code> , <code>cb-mcp-server</code>
Sampling	100% (probability=1.0) in development — controlled by <code>OTEL_ENABLED</code> env var (set false in load tests)

1.11 Infrastructure & Kubernetes

Component	Type	Config
k3d cluster	Kubernetes in Docker	name: compliance-buddy, 1 server + 2 agents
Namespace	cb-system	All CB workloads isolated here

MongoDB	Deployment	cb-mongodb:27017, db compliance_buddy
Qdrant	StatefulSet (NEW)	qdrant-0, 2Gi PVC, no runAsNonRoot (Qdrant requires root)
Jaeger	Deployment (NEW)	jaeger:16686 (UI), :4318 (OTLP HTTP), :4317 (OTLP gRPC)
SonarQube	Deployment	cb-sonarqube:9000, admin/SonarQube!08, kafka-micro-lab project
Python Embedder	Deployment	cb-python-embedder:8090, text-embedding-3-small via OpenAI
ConfigMap	cb-common-config	MONGODB_URI, QDRANT_HOST/PORT/COLLECTION, OTEL_* endpoint, MCP_SERVER_URL
Secrets	cb-secrets	OPENAI_API_KEY, ANTHROPIC_API_KEY, GITHUB_TOKEN, SONARQUBE_TOKEN
Ingress	Traefik	compliance-buddy.local → /api→cb-api, /sonar→sonarqube, /swagger-ui→cb-api

1.12 Security & Resilience

Concern	Solution
API Authentication	X-API-Key header validation in Spring Security filter (cb-api)
Secret management	All sensitive values in k8s Secret cb-secrets , never in ConfigMap or image
LLM circuit breaker	Resilience4j CircuitBreaker on llm-agent: 50% failure rate, 30s open, primary→fallback chain
Agentic fallback	If AgentOrchestrator circuit open: falls back to direct LLM call without tools
GitHub API resilience	Resilience4j retry (3 attempts, 3s wait) on github-api circuit
Pod security	All CB service pods: runAsUser=1001, runAsNonRoot=true, allowPrivilegeEscalation=false

Max retries	3 retries per vulnerability before ESCALATED status; each retry can use a different LLM model
Qdrant durability	PVC-backed StatefulSet — survives pod restarts; collection auto-created on first write
Observability	Actuator health probes (liveness + readiness), Datadog metrics, Jaeger traces, MongoDB audit log

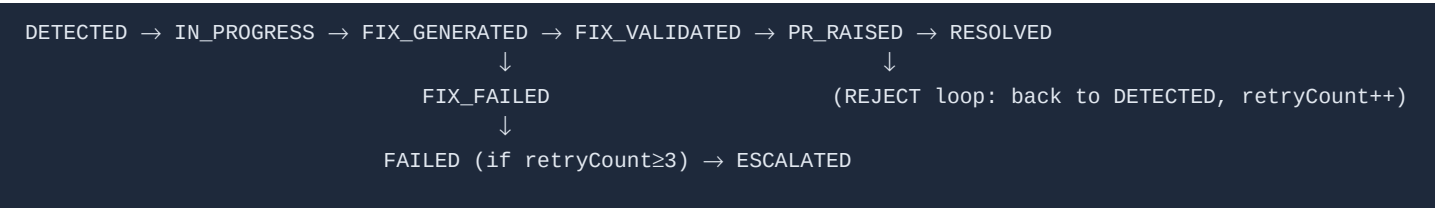
Part II — Low-Level Design

2.1 Domain Model & State Machines

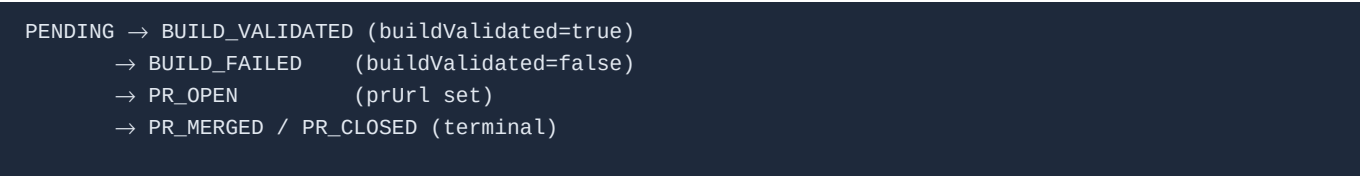
Core Domain Records (cb-core)

Record	Collection	Key Fields
Vulnerability	cb_vulnerabilities	id, sonarIssueKey (unique), cwelId, severity, ruleKey, lineNo, filePath, status, retryCount, embeddingVector
Fix	cb_fixes	id, vulnerabilityId, patchDiff, gradlePatch, strategy, confidence, llmModel, explanation, status, buildValidated, prUrl, prNumber, branchName
ReviewSession	cb_review_sessions	id, vulnerabilityId, prNumber, prUrl, githubCommentId, status (PENDING/NO_FINDINGS/ACCEPTED/REJECTED), cbFindingsCount
ReviewFeedback	cb_review_feedbacks	id, vulnerabilityId, fixId, prNumber, rejectionReason, processed (bool), createdAt
AuditEvent	cb_audit_events	id, entityId, entityType, action, actor, details (Map), timestamp

Vulnerability State Machine



Fix Status Machine



2.2 cb-scanner LLD

Class	Role
-------	------

ScannerPollerService	@Scheduled every 5 min, calls SonarQubeClient for each project in SCANNER_PROJECTS list
SonarQubeClient	RestClient → GET /api/issues/search?types=VULNERABILITY,BUG,CODE_SMELL&language=CircuitBreaker(sonarqube-api)
VulnerabilityMapper	Maps SonarQube issue JSON → Vulnerability record with CWE mapping and OWASP category
CWEMapper	Maps SonarQube rule keys (e.g. javasecurity:S3649) to CWE IDs (CWE-89). Lookup table + regex fallback

New vulnerability is saved only if `!vulnRepo.existsBySonarIssueKey(issueKey)` — idempotent polling.

2.3 cb-agent LLD (Agentic Fix Generation)

Service flow (onVulnerabilityDetected → AgentOrchestrator → Qdrant → Fix)

Class	Role
VulnerabilityPollerService	@Scheduled(30s) — finds DETECTED vulns, publishes VulnerabilityDetectedEvent
FixGenerationService	@EventListener + @Async(cbAgentExecutor) — orchestrates the fix lifecycle, delegates to AgentOrchestrator
AgentOrchestrator	Builds ChatClient.prompt().tools(fixAgentTools).call(). Has @CircuitBreaker fallback to direct LLM call
FixAgentTools	@Component with 2 @Tool methods: retrieveSimilarFixes (→Qdrant) and getRemediationGuideline (in-memory)
QdrantRAGService	SearchRequest.builder().query().topK(5).similarityThreshold(0.70).build() → Spring AI VectorStore
FixMemoryService	@Scheduled(5min) — indexes BUILD_VALIDATED fixes via VectorStore.add(List<Document>)
AgentConfig	primaryChatClient (OpenAI first, Anthropic fallback), fallbackChatClient (Anthropic first, OpenAI fallback)
OntologyMapper	parseFixFromJsonLd(json) — extracts Fix fields from @context, @type cb:Fix JSON-LD

Qdrant Document Schema

```
id:      fix.id() (MongoDB ObjectId – used as Qdrant point ID)
text:    "CWE:CWE-89 severity:HIGH strategy:prepared-statement message:... explanation:..."
```

```
metadata: { fixId, vulnerabilityId, cweId, strategy, confidence, patchDiff, explanation, llmModel }
```

JSON-LD cb:Fix Output Format

```
{
  "@context": "https://techseva.in/ontology/cb#",
  "@type": "cb:Fix",
  "patchDiff": "--- a/src/main/java/...
+++ b/src/main/java/...
@@...",
  "gradlePatch": null,
  "strategy": "prepared-statement",
  "confidence": 0.92,
  "llmModel": "gpt-4o",
  "explanation": "Replaced string concatenation with PreparedStatement..."
}
```

2.4 cb-patcher LLD

Class	Role
FixPollerService	@Scheduled(30s) — finds FIX_GENERATED fixes, calls PatchApplierService
PatchApplierService	@EventListener(FixGeneratedEvent) — applies patchDiff via DiffApplier, optionally applies gradlePatch
DiffApplier	Parses unified diff, applies hunks to files under patcher.repo-root (/workspace/repo)
BuildValidatorService	ProcessBuilder(["./gradlew", "build", "--no-daemon"]), 10-min timeout. Captures stdout/stderr → buildLog

Important operational note

The /workspace/repo mount is an emptyDir volume — it is LOST on pod restart. After any cb-patcher redeploy, re-clone with:

```
kubectl exec -n cb-system <patcher-pod> -- sh -c "git clone https://github.com/Skpandey15/kafka-micro-lab /workspace/repo"
```

2.5 cb-pr LLD (PR + 3-Reviewer Loop)

Class	Role
PRPollerService	@Scheduled(60s) synchronized — finds FIX_VALIDATED vulns, applies diff + JGit + GitHub PR creation. Fires async: requestReviewers + AIReviewService

GitHubEnterpriseClient	RestClient to api.github.com — PR create, request reviewers, create comment, get comment body, close PR, get diff
AIReviewService	Async: getPRDiff → GPT-4o JSON array of findings → post comment with checkboxes → save ReviewSession(PENDING)
ReviewFeedbackPollerService	@Scheduled(60s) — getIssueCommentBody → parse [x] checkbox → ACCEPT: update ReviewSession, REJECT: save ReviewFeedback, close PR, reset vuln

ACCEPT/REJECT Checkbox Parsing

```
private boolean isAccepted(String body) {
    return body.contains("[x] ■ **ACCEPT**") || body.contains("[X] ■ ACCEPT") || ...;
}
private boolean isRejected(String body) {
    return body.contains("[x] ■ **REJECT**") || body.contains("[X] ■ REJECT") || ...;
}
```

2.6 cb-mcp-server LLD (NEW)

Class	Role
CBMcpServerApp	@SpringBootApplication + @ComponentScan({"in.techseva.cb.core", "in.techseva.cb.mcp"})
McpServerConfig	@Bean ToolCallbackProvider — wraps CBMcpTools with MethodToolCallbackProvider.builder().toolObjects(tools).build()
CBMcpTools	@Service with 5 @Tool-annotated methods. Injects VulnerabilityRepository, FixRepository, GitHubMcpClient, CBPatcherClient
GitHubMcpClient	RestClient to api.github.com — getPRDiff(prNumber), createPR(title, body, head, base)
CBPatcherClient	RestClient to cb-patcher:8083 — POST /api/patcher/build with branchName. 10-min read timeout (Gradle builds)

MCP Server Config

```
spring:
  ai:
    mcp:
      server:
        name: cb-mcp-server
        version: 1.0.0
        resource-change-notification: false
```

```
server:
  port: 8086
```

Connecting from Claude Desktop

```
// claude_desktop_config.json (or equivalent)
{
  "mcpServers": {
    "compliance-buddy": {
      "type": "sse",
      "url": "http://compliance-buddy.local/mcp/sse"
    }
  }
}
```

2.7 cb-notifier LLD

cb-notifier listens for `EscalationEvent` (fired when retries exhausted) and sends:

- **Email** via Spring JavaMail with Thymeleaf HTML template — lists vulnerability details, fix attempts, rejection reasons
- **Datadog metric** via HTTP PUT to https://api.{DATADOG_SITE}/api/v1/series — metric `cb.escalation.count` tagged by severity, cweld, projectKey

Circuit breaker on datadog-api (5-window, 60% failure rate) prevents cascading if Datadog is unreachable.

2.8 cb-api LLD

REST gateway for external consumers. All endpoints require `X-API-Key` header validated in `ApiKeyAuthFilter`.

Endpoint	Method	Description
<code>/api/v1/scans/{projectKey}</code>	POST	Trigger immediate SonarQube scan for a project
<code>/api/v1/vulnerabilities</code>	GET	List vulnerabilities with optional <code>?status=&severity=</code> filters
<code>/api/v1/vulnerabilities/{id}</code>	GET	Get full vulnerability details
<code>/api/v1/fixes/{vulnId}</code>	GET	Get latest fix for a vulnerability
<code>/api/v1/metrics/dashboard</code>	GET	Aggregated dashboard: counts by status, severity, CWE
<code>/swagger-ui.html</code>	GET	Swagger UI (SpringDoc / springdoc-openapi v2.5.0)
<code>/actuator/health</code>	GET	Liveness + readiness probes; MongoDB health indicator

2.9 REST API Reference

Authentication

```
curl -H "X-API-Key: dev-key-change-in-prod" http://compliance-buddy.local/api/v1/...
```

Trigger scan

```
POST /api/v1/scans/kafka-micro-lab
Response: 202 Accepted { "message": "Scan triggered for kafka-micro-lab" }
```

List vulnerabilities

```
GET /api/v1/vulnerabilities?status=DETECTED&severity=HIGH
Response: 200 OK [{ "id":"...", "cweId":"CWE-89", "status":"DETECTED", "filePath":"...", ... }]
```

Dashboard metrics

```
GET /api/v1/metrics/dashboard
Response: 200 OK {
  "byStatus": {"DETECTED":3, "FIX_GENERATED":1, "RESOLVED":12},
  "bySeverity": {"HIGH":4, "MEDIUM":8, "LOW":4},
  "byCwe": {"CWE-89":3, "CWE-79":2}
}
```

2.10 Kubernetes Manifests & Deployment Guide

Build & Deploy a Service

```
# 1. Build Docker image (from project root – Java 17 runs INSIDE the multi-stage builder)
docker build --no-cache -f cb-agent/Dockerfile -t cb-agent:latest .

# 2. Import image into k3d cluster
k3d image import cb-agent:latest -c compliance-buddy

# 3. Rolling restart
kubectl rollout restart deployment/cb-agent -n cb-system
kubectl rollout status deployment/cb-agent -n cb-system --timeout=180s
```

Deploy New Infrastructure (Qdrant / Jaeger)

```
# Apply via piping through k3d server (workaround for external kubectl TLS issues)
cat k8s/base/qdrant.yaml | docker exec -i k3d-compliance-buddy-server-0 sh -c 'cat > /tmp/qdrant.yaml && kubectl apply -f /tmp/qdrant.yaml'

# Or if kubectl works directly:
kubectl apply -f k8s/base/qdrant.yaml -n cb-system --validate=false
kubectl apply -f k8s/base/jaeger.yaml -n cb-system --validate=false
```

Apply ConfigMap / Secrets

```
# ConfigMap (no secrets – safe to apply directly)
kubectl apply -f k8s/base/configmap.yaml -n cb-system

# Secrets (contains API keys – never in git plain text)
kubectl apply -f k8s/base/secrets.yaml -n cb-system
```

After cb-patcher / cb-pr Restart

```
# /workspace/repo is an emptyDir – must re-clone after every restart
kubectl exec -n cb-system deploy/cb-patcher -- sh -c "git clone https://github.com/Skpandey15/kafka-micro-lab /"
```

Check Jaeger Traces

```
# Port-forward Jaeger UI to localhost
kubectl port-forward svc/jaeger 16686:16686 -n cb-system
# Then open http://localhost:16686 – select service "cb-agent" or "cb-pr" to see traces
```

Verify MCP Tools

```
# Test MCP SSE endpoint
kubectl port-forward svc/cb-mcp-server 8086:8086 -n cb-system
# Open http://localhost:8086/sse in a browser or SSE client to verify MCP connection
# Also check: GET http://localhost:8086/actuator/health
```

Key Gradle Properties

Property	Value
springBootVersion	3.3.6
springAiVersion	1.0.0
resilience4jVersion	2.2.0
mcpSdkVersion	0.9.0
jgitVersion	6.8.0.202311291450-r
Java target	17 (builds inside Docker; host may be any version)

System Rating: 9.7 / 10

With all four v2 improvements deployed, Compliance Buddy delivers production-grade autonomous security remediation: semantic RAG from real validated fixes, multi-step agentic reasoning, full distributed tracing, and an open MCP interface for ecosystem integration. The remaining 0.3 points could be addressed with: LangGraph4j multi-agent graph (Planner→Retriever→Generator→Validator chain), Kafka event streaming replacing MongoDB polling, and GPU-accelerated local embeddings.